Noteflow: Project Setup and FastAPI Backend Integration Guide

Oct 13, 2026 · @Mani

Overview

Noteflow is a study-notes app built with Next.js (App Router), React 19 and Tailwind CSS v4, split out of a single `page.tsx` into separate folders and files. Today it runs entirely on in-memory mock state. This guide covers how to run it with pnpm, how the code is organized, and what to add when you connect a FastAPI backend.

Running with pnpm

Unzip the project, then install and start the dev server. Nothing in the project is npm-specific, so `pnpm install` will create `pnpm-lock.yaml` on first run.

```
unzip noteflow.zip && cd noteflow
pnpm install
pnpm dev        # http://localhost:3000
```

The other scripts are `pnpm build`, `pnpm start` and `pnpm typecheck`.

On pnpm 10 or newer, the install may print "Ignored build scripts" for packages like `sharp` or `@tailwindcss/oxide`. The app still runs without them: Next only uses `sharp` for image optimization, which this project doesn't use, and Tailwind ships prebuilt binaries. To approve them anyway, run `pnpm approve-builds`.

Current project structure

Routes live in `src/app/` and only render components; the UI lives in `src/components/`, grouped by feature.

Folder	What it holds
`src/app/`	Routes only. `(auth)` has login and register; `(app)` has dashboard, notes (list, new, `[id]`, `[id]/edit`), quiz (results, generating) and account (settings, delete).
`src/components/ui/`	Button, Badge, Brand, Icon, Modal, PageHeading, PasswordInput, BackLink.
`src/components/layout/`	AppShell, AuthShell, Sidebar, Header, MobileNav.
`src/components/dashboard/`, `notes/`, `quiz/`, `auth/`, `account/`, `api-key/`	Feature components for each area.
`src/context/app-context.tsx`	Shared state: notes, profile, API key, delete-note modal.
`src/data/`	Seed notes, subjects, quiz text, chart values.
`src/lib/`	The `ui` class alias map, `cn()`, routes, nav helpers, utils.
`src/types/`	Shared types (`Note`, `UserProfile`).

Styling is unchanged from the original file. `font-['Newsreader']` became `font-heading`, wired to `next/font` in `globals.css`.

Files and folders to add for FastAPI

You mainly add an API layer and a few config files, then replace the mock state in `app-context.tsx`.

```
noteflow/
├─ .env.local                        # API_URL=http://localhost:8000  (server-
only)
├─ next.config.ts                    # edit: add rewrites to FastAPI (optional)
└─ src/
   ├─ proxy.ts                       # route protection (Next 16 name for
middleware)
   ├─ lib/api/
   |  ├─ client.ts                   # fetch wrapper: base URL, cookies, error
handling
   |  ├─ auth.ts                     # login, register, logout, me, delete
account
   |  ├─ notes.ts                    # list, get, create, update, delete
   |  └─ quiz.ts                     # generate quiz, get quiz
   ├─ types/api.ts                   # request/response types (or generate from
/openapi.json)
   ├─ hooks/                         # only if you use TanStack Query
   |  ├─ use-notes.ts
   |  └─ use-quiz.ts
   ├─ providers/query-provider.tsx # only if you use TanStack Query
   └─ app/(app)/
      ├─ loading.tsx                 # skeleton while data loads
      ├─ error.tsx                   # backend-down / failed request UI
      └─ notes/[id]/not-found.tsx  # real 404 once notes come from the server
```

Key decisions

Decision	Recommended approach
CORS vs. rewrites	Add a rewrite in `next.config.ts` that sends `/api/*` to FastAPI, so the browser only talks to Next.js and you avoid CORS. Otherwise, add `CORSMiddleware` on the FastAPI side with credentials allowed.
Auth	Have FastAPI set an httpOnly session cookie (or a JWT in a cookie), and let `proxy.ts` redirect to `/login` when it's missing. Don't keep tokens in `localStorage` or in the React context.

Decision	Recommended approach
Data fetching	Notes and the dashboard can become server components that call `lib/api/*` directly. Interactive parts (forms, delete modal) can use server actions or TanStack Query.
OpenAI key	Keep it on the FastAPI server and drop the key-entry modal. "Generate quiz" should call something like `POST /notes/{id}/quiz`, and the progress screen can poll for status.

Existing files to edit or remove

File	Change
`context/app-context.tsx`	Replace the in-memory `notes`, `profile` and `apiConfigured` with real data. It can shrink to just UI state (delete modal) or go away.
`login-form.tsx`, `register-form.tsx`, `delete-account-view.tsx`, `note-form.tsx`	Replace the `setTimeout` fakes with API calls.
`quiz-progress.tsx`, `quiz-results.tsx`	Replace the static text with the backend response.
`api-key-modal.tsx` and the `apiConfigured` checks	Remove or repurpose once the key lives on the server.
`data/*`	Delete the seed data once the backend is the source.

Current limitations

- Login, notes and the API key are in-memory mock state, as in the original file. A refresh resets them and nothing protects the routes.

- The project was not built or type-checked when it was generated, because there was no network access. Only syntax, imports and class-name aliases were checked, so run `pnpm dev` and `pnpm typecheck` and report any errors.
- The original global stylesheet wasn't available, so `globals.css` is minimal. Small decorative bits that relied on it, like status dots and the spinner, may look different or be invisible.